

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторные работы №2 и №3 по курсу «Дискретный анализ»

Студент: М. А. Хорохорин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторные работы №2 и №3

Лабораторная работа №2

Вариант структуры данных: PATRICIA.

Необходимо создать программную библиотеку, реализующую словарь на основе дерева PATRICIA. В словаре каждому ключу, представляющему собой регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие номер от 0 до $2^{64} - 1$.

Программа должна поддерживать добавление, удаление, поиск, сохранение словаря в бинарном виде и загрузку словаря из файла. При ошибках ввода-вывода или повреждённом формате внешнего файла программа должна печатать строку, начинающуюся с `ERROR:`, а рабочий словарь должен оставаться в корректном состоянии.

Лабораторная работа №3

Для реализации словаря из лабораторной работы №2 необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов требуется их исправить.

Результатом лабораторной работы является отчёт, содержащий дневник выполнения работы, выводы о найденных недочётах, сравнение исправленной программы с предыдущей версией и общие выводы о выполненной работе.

Для исследования использовались:

- `gprof` — профилирование времени выполнения;
- `Valgrind Memcheck` — проверка ошибок работы с памятью и утечек;
- `Valgrind Massif` — оценка пикового потребления памяти;
- собственный генератор тестов и `benchmark` из лабораторной работы №2.

1 Описание

Требуется написать реализацию словаря на основе дерева PATRICIA (Practical Algorithm To Retrieve Information Coded In Alphanumeric), а затем провести исследование качества полученной программы.

PATRICIA-дерево — это сжатое бинарное trie, в котором каждый узел хранит номер критического бита ключа, по которому происходит ветвление. Это позволяет избежать цепочек узлов с одним потомком и хранить только те позиции, в которых ключи действительно расходятся.

Ключевая особенность структуры — *обратные рёбра*: когда при обходе номер критического бита следующего узла не превышает номера текущего, обход заканчивается. Узел назначения такого ребра является кандидатом для полного сравнения строк.

Для корректной обработки граничных случаев используется узел-заголовок (**header**) с нулевым критическим битом, левый потомок которого является корнем дерева. Все ключи переводятся в нижний регистр, что обеспечивает регистронезависимость. Словарь поддерживает операции **Insert**, **Find**, **Remove**, а также бинарные **Save** и **Load**.

2 Исходный код

1 Вспомогательные функции

Функция `ExtractBit` извлекает i -й бит строки (нумерация с 1, старший бит первого байта — бит 1). Функция `FindFirstDiffBit` находит позицию первого бита, в котором два ключа различаются, — результат используется при вставке для определения критического бита нового узла. Функция `NormalizeKey` приводит ключ к нижнему регистру.

Листинг 1: Вспомогательные функции

```
1 static int ExtractBit(const std::string &key, int pos) {
2     int byteIdx = (pos - 1) / 8;
3     int bitOff = 7 - ((pos - 1) % 8);
4     if (byteIdx >= static_cast<int>(key.size())) {
5         return 0;
6     }
7     return (static_cast<unsigned char>(key[byteIdx]) >> bitOff) & 1;
8 }
9
10 static std::string NormalizeKey(const std::string &s) {
11     std::string result(s.size(), '\0');
12     for (int i = 0; i < static_cast<int>(s.size()); i++) {
13         result[i] = static_cast<char>(
14             tolower(static_cast<unsigned char>(s[i])));
15     }
16     return result;
17 }
18
19 static int FindFirstDiffBit(const std::string &a, const std::string &b) {
20     int maxLen = static_cast<int>(
21         a.size() > b.size() ? a.size() : b.size());
22     for (int i = 0; i < maxLen; i++) {
23         unsigned char x = (i < static_cast<int>(a.size()))
24             ? static_cast<unsigned char>(a[i]) : 0;
25         unsigned char y = (i < static_cast<int>(b.size()))
26             ? static_cast<unsigned char>(b[i]) : 0;
27         if (x != y) {
28             unsigned char diff = x ^ y;
29             int shift = 0;
30             while ((diff & 0x80) == 0) { diff <= 1; shift++; }
31             return i * 8 + shift + 1;
32         }
33     }
34     return maxLen * 8 + 1;
35 }
```

2 Структура узла и класс дерева

Каждый узел хранит ключ, значение, номер критического бита и указатели на двух потомков. Поле `size` в классе `TPatriciaTrie` отслеживает количество элементов.

Листинг 2: Объявления `TNode` и `TPatriciaTrie`

```
1 struct TNode {
2     std::string key;
3     uint64_t value;
4     int checkBit;
5     TNode *left;
6     TNode *right;
7
8     TNode(const std::string &k, uint64_t v, int b);
9 };
10
11 class TPatriciaTrie {
12 public:
13     TPatriciaTrie();
14     ~TPatriciaTrie();
15
16     bool Find(const std::string &key, uint64_t &value) const;
17     bool Insert(const std::string &key, uint64_t value);
18     bool Remove(const std::string &key);
19     bool Save(const std::string &path) const;
20     bool Load(const std::string &path);
21
22 private:
23     TNode *header;
24     int size;
25
26     TNode *SearchNode(const std::string &key) const;
27     TNode **FindParentPtr(TNode *target, const std::string &key) const;
28     void SaveSubtree(std::ofstream &file, TNode *cur) const;
29     void DestroySubtree(TNode *cur);
30 };
```

3 Поиск

Функция `SearchNode` спускается по дереву, выбирая ветвь по критическому биту, до тех пор пока не встретит обратное ребро. Найденный узел является только кандидатом: `Find` дополнительно выполняет полное сравнение строк.

Листинг 3: `SearchNode` и `Find`

```
1 TNode *TPatriciaTrie::SearchNode(const std::string &key) const {
```

```

2 | TNode *prev = header;
3 | TNode *cur = header->left;
4 | while (cur->checkBit > prev->checkBit) {
5 |     prev = cur;
6 |     if (ExtractBit(key, cur->checkBit)) {
7 |         cur = cur->right;
8 |     } else {
9 |         cur = cur->left;
10 |    }
11 | }
12 | return cur;
13 | }
14 |
15 | bool TPatriciaTrie::Find(const std::string &key, uint64_t &value) const {
16 |     if (!header) {
17 |         return false;
18 |     }
19 |     TNode *found = SearchNode(NormalizeKey(key));
20 |     if (found->key != NormalizeKey(key)) {
21 |         return false;
22 |     }
23 |     value = found->value;
24 |     return true;
25 | }

```

4 Вставка

При вставке сначала выполняется `SearchNode`, чтобы найти коллизию или определить ключ-ориентир. Затем `FindFirstDiffBit` вычисляет критический бит нового узла. После этого выполняется повторный спуск до позиции вставки, и новый узел подвешивается с обратным ребром на самого себя.

Листинг 4: Insert

```

1 | bool TPatriciaTrie::Insert(const std::string &key, uint64_t value) {
2 |     std::string lkey = NormalizeKey(key);
3 |     if (!header) {
4 |         header = new TNode(lkey, value, 0);
5 |         header->left = header;
6 |         header->right = nullptr;
7 |         size++;
8 |         return true;
9 |     }
10 |
11 |     TNode *existing = SearchNode(lkey);
12 |     if (existing->key == lkey) {
13 |         return false;

```

```

14     }
15
16     int diffBit = FindFirstDiffBit(lkey, existing->key);
17     TNode *prev = header;
18     TNode *cur = header->left;
19     while (cur->checkBit > prev->checkBit && cur->checkBit < diffBit) {
20         prev = cur;
21         if (ExtractBit(lkey, cur->checkBit)) {
22             cur = cur->right;
23         } else {
24             cur = cur->left;
25         }
26     }
27
28     TNode *newNode = new TNode(lkey, value, diffBit);
29     if (ExtractBit(lkey, diffBit)) {
30         newNode->right = newNode;
31         newNode->left = cur;
32     } else {
33         newNode->left = newNode;
34         newNode->right = cur;
35     }
36
37     if (prev == header) {
38         header->left = newNode;
39     } else if (ExtractBit(lkey, prev->checkBit)) {
40         prev->right = newNode;
41     } else {
42         prev->left = newNode;
43     }
44     size++;
45     return true;
46 }

```

5 Удаление

Удаление обрабатывает два случая. Если у удаляемого узла есть обратное ребро на самого себя, родительский указатель переключается на другого потомка. Иначе находится узел, который содержит обратное ребро на удаляемый элемент; его данные переносятся в удаляемый узел, а сам найденный узел физически удаляется.

Листинг 5: Remove — два случая удаления

```

1 bool TPatriciaTrie::Remove(const std::string &key) {
2     if (!header) {
3         return false;
4     }

```

```

5
6     std::string lkey = NormalizeKey(key);
7     TNode *target = SearchNode(lkey);
8     if (target->key != lkey) {
9         return false;
10    }
11
12    bool selfRef = (target->left == target || target->right == target);
13    if (selfRef) {
14        if (target == header) {
15            delete header;
16            header = nullptr;
17            size--;
18            return true;
19        }
20        TNode *other = (target->left == target) ? target->right : target->left;
21        *FindParentPtr(target, lkey) = other;
22        delete target;
23        size--;
24        return true;
25    }
26
27    /* ... find back-edge holder, move its data, fix links ... */
28    size--;
29    return true;
30 }

```

6 Сериализация

Метод **Save** обходит дерево рекурсивно и записывает каждый узел как тройку: длина ключа, сам ключ и значение. Длина хранится в `uint16_t`, потому что по условию ключ может иметь длину 256 символов. Метод **Load** читает записи во временное дерево, проверяет корректность каждой записи и заменяет текущее состояние только после успешного чтения всего файла.

Листинг 6: SaveSubtree и Load

```

1 void TPatriciaTrie::SaveSubtree(std::ofstream &file, TNode *cur) const {
2     uint16_t klen = static_cast<uint16_t>(cur->key.size());
3     file.write(reinterpret_cast<const char *>(&klen), sizeof(klen));
4     file.write(cur->key.data(), klen);
5     file.write(reinterpret_cast<const char *>(&cur->value),
6               sizeof(cur->value));
7     if (cur->left && cur->left->checkBit > cur->checkBit) {
8         SaveSubtree(file, cur->left);
9     }
10    if (cur->right && cur->right->checkBit > cur->checkBit) {

```

```

11     SaveSubtree(file, cur->right);
12 }
13 }
14
15 bool TPatriciaTrie::Load(const std::string &path) {
16     std::ifstream file(path, std::ios::binary);
17     if (!file) {
18         return false;
19     }
20
21     TPatriciaTrie tmp;
22     while (true) {
23         uint16_t klen = 0;
24         if (!file.read(reinterpret_cast<char *>(&klen), sizeof(klen))) {
25             if (!file.eof()) { return false; }
26             break;
27         }
28         if (klen == 0 || klen > MAX_KEY_LENGTH) {
29             return false;
30         }
31         std::string key(klen, '\0');
32         if (!file.read(&key[0], klen)) { return false; }
33
34         uint64_t val = 0;
35         if (!file.read(reinterpret_cast<char *>(&val), sizeof(val))) {
36             return false;
37         }
38         if (!tmp.Insert(key, val)) {
39             return false;
40         }
41     }
42     std::swap(header, tmp.header);
43     std::swap(size, tmp.size);
44     return true;
45 }

```

7 Обработка команд

Функция `main` читает команды построчно. Добавление, удаление, поиск, сохранение и загрузка выводят ответы в формате условия.

Листинг 7: Обработка команд в `main`

```

1 while (std::getline(std::cin, line)) {
2     std::istringstream ss(line);
3     std::string cmd;
4     ss >> cmd;
5     if (cmd == "+") {

```

```

6         std::string word;
7         uint64_t val;
8         ss >> word >> val;
9         std::cout << (trie.Insert(word, val) ? "OK" : "Exist") << "\n";
10    } else if (cmd == "-") {
11        std::string word;
12        ss >> word;
13        std::cout << (trie.Remove(word) ? "OK" : "NoSuchWord") << "\n";
14    } else if (cmd == "!") {
15        std::string subcmd, path;
16        ss >> subcmd >> path;
17        if (subcmd == "Save") {
18            std::cout << (trie.Save(path) ? "OK" : "ERROR: cannot save") << "\n";
19        } else if (subcmd == "Load") {
20            std::cout << (trie.Load(path) ? "OK" : "ERROR: cannot load") << "\n";
21        }
22    } else {
23        uint64_t val;
24        if (trie.Find(cmd, val)) {
25            std::cout << "OK: " << val << "\n";
26        } else {
27            std::cout << "NoSuchWord\n";
28        }
29    }
30 }

```

3 Консоль

```
root@container:/workspace# cmake -B build -DCMAKE_BUILD_TYPE=Release
root@container:/workspace# cmake --build build
root@container:/workspace# ./build/lab2/lab2_main
+ a 1
OK
+ A 2
Exist
A
OK: 1
-A
OK
a
NoSuchWord
```

4 Результаты исследования производительности

В данном разделе представлены результаты тестирования PATRICIA-дерева в сравнении со стандартным контейнером `std::map`, а также анализ потребления памяти и профилирования исправленной программы.

1 Анализ потребления оперативной памяти

Для проверки ошибок работы с динамической памятью использовалась утилита Valgrind Memcheck. Запуск выполнялся на наборе из 100 000 вставок, 100 000 поисков и 100 000 удалений.

HEAP SUMMARY:

in use at exit: 122,880 bytes in 6 blocks

total heap usage: 261,459 allocs, 261,453 frees, 9,988,494 bytes allocated

LEAK SUMMARY:

definitely lost: 0 bytes in 0 blocks

indirectly lost: 0 bytes in 0 blocks

possibly lost: 0 bytes in 0 blocks

still reachable: 122,880 bytes in 6 blocks

ERROR SUMMARY: 0 errors from 0 contexts

Показатель	PATRICIA	Комментарий
Ошибки памяти	0	Некорректных чтений и записей не обнаружено
Утечки пользовательской памяти	0 bytes	Узлы дерева освобождаются корректно
<code>still reachable</code>	122880 bytes	Буферы стандартной библиотеки ввода-вывода
Пик heap по Massif	7.335 MB	Набор из 100 000 слов

Вывод по памяти: утечек в пользовательском коде не найдено. Память, отмеченная как `still reachable`, относится к внутренним буферам `libstdc++` и не является ошибкой реализации словаря.

2 Исследование скорости выполнения

Замеры времени проводились на операциях вставки, поиска и удаления строковых ключей. Сравнение выполнялось с `std::map`.

N	PATRICIA, мкс			std::map, мкс		
	Insert	Find	Remove	Insert	Find	Remove
1 000	348	208	465	214	161	303
10 000	4 731	3 246	8 264	4 251	2 405	3 711
100 000	86 469	75 838	100 506	87 293	73 691	81 991
500 000	731 836	695 895	764 450	766 442	748 168	785 446
1 000 000	1 842 835	2 022 130	2 038 648	1 953 836	1 985 049	2 322 171

На малых объёмах `std::map` быстрее из-за меньших накладных расходов. На больших наборах PATRICIA начинает выигрывать на части операций: при $N = 500\,000$ быстрее поиск и удаление, при $N = 1\,000\,000$ быстрее вставка и удаление.

3 Анализ производительности с помощью gprof

Для детального анализа времени выполнения была собрана цель `lab3_profile` с флагом `-pg`. Отчёт `gprof` для 100 000 операций каждого типа показал следующие основные строки.

Функция	Доля времени, %	Число вызовов
<code>main</code>	46.68	—
<code>TPatriciaTrie::Remove</code>	20.01	100 000
<code>TPatriciaTrie::Find</code>	17.78	100 000
<code>TPatriciaTrie::Insert</code>	15.56	100 000
<code>NormalizeKey</code>	0.00	400 000

Профилирование показывает, что основное время тратится на разбор входа и операции дерева. Удаление является наиболее дорогой операцией PATRICIA, потому что после физического удаления узла нужно корректно восстановить обратные рёбра.

4 Сравнение исходной и исправленной версии

Версия	Insert, мкс	Find, мкс	Remove, мкс
Исходная	111 895	111 360	128 464
Исправленная	99 566	109 004	116 582

Исправления не изменили асимптотику операций дерева. Разница во времени находится в пределах практических колебаний запусков внутри контейнера; основная польза исправлений заключается в корректной обработке ошибок сериализации и честном benchmark.

5 Дневник выполнения работы

- Этап 1: Изучение PATRICIA-дерева. Разбор критических битов, обратных рёбер и узла-заголовка.
- Этап 2: Реализация словаря. Добавлены операции `Insert`, `Find`, `Remove`, регистронезависимая обработка ключей и бинарная сериализация.
- Этап 3: Подготовка `benchmark` и генератора. Проведено сравнение PATRICIA с `std::map` на наборах от 1 000 до 1 000 000 ключей.
- Этап 4: Исследование качества. Использованы `gprof`, `Valgrind Memcheck` и `Valgrind Massif`; получены данные по времени выполнения и памяти.
- Этап 5: Исправление недочётов. Исправлены ошибки загрузки файла, проверки бинарного формата, сохранения ключа длиной 256 и замера поиска в `benchmark`.
- Этап 6: Пересборка отчёта и контрольная проверка актуальности результатов.

6 Выявленные недочёты и исправления

В ходе исследования были устранены следующие проблемы:

1. Ошибка загрузки отсутствующего файла: метод `Load` возвращал успех, если файл нельзя было открыть. Исправлено: теперь возвращается ошибка, а программа печатает `ERROR: cannot load`.
2. Повреждённый бинарный файл мог быть принят частично. Исправлено: загрузка выполняется во временное дерево, проверяется полнота каждой записи, а рабочий словарь заменяется только после успешного чтения всего файла.
3. Длина ключа сохранялась в `uint8_t`. Для ключа длиной 256 символов происходило переполнение. Исправлено: длина хранится в `uint16_t`.
4. `Benchmark` не фиксировал результат поиска `PATRICIA`. Исправлено: добавлен `volatile`-счётчик успешных поисков, поэтому компилятор не может удалить цикл измерения.

7 Выводы

В ходе выполнения лабораторных работ была реализована структура данных PATRICIA для словаря строковых ключей и проведено исследование качества полученной программы. Реализованы поиск, вставка, удаление, сохранение и загрузка словаря из бинарного файла.

Профилирование с помощью `gprof` показало, что основное время расходуется в операциях `Remove`, `Find` и `Insert`, что соответствует устройству программы: именно эти функции выполняют обход дерева по критическим битам.

Проверка `Valgrind Memcheck` не обнаружила утечек памяти и некорректных обращений в коде словаря. `Massif` показал умеренное потребление памяти: около 7.335 МБ heap-памяти для набора из 100 000 слов.

Были исправлены функциональные недочёты сериализации: загрузка отсутствующего файла, повреждённого файла и ключа максимальной длины 256 символов. Также был исправлен `benchmark`, чтобы измерение поиска PATRICIA не могло быть сведено оптимизатором к пустому циклу.

Главный результат работы — программа стала надёжнее на ошибочных входах и честнее измеряется в тестах производительности, при этом базовая структура данных и её асимптотические свойства остались прежними.

8 Список литературы

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*. MIT Press.
2. Donald R. Morrison. PATRICIA — Practical Algorithm To Retrieve Information Coded In Alphanumeric. *Journal of the ACM*, 1968.
3. GNU gprof documentation. <https://sourceware.org/binutils/docs/gprof/>
4. Valgrind User Manual. <https://valgrind.org/docs/manual/>